

Python Functional Toolkit

lambda • map() • filter() • reduce()

λ

$\Rightarrow$

$\oplus$

$\emptyset$

Σ

A quick visual reference guide with runnable examples and expected output

λ Lambda Functions

A **lambda function** is a small, anonymous (unnamed) function in Python. It's written in a single line and is ideal for short, throwaway logic — especially when used alongside `map()`, `filter()`, and `reduce()`.

def	lambda
Has a function name	Anonymous (can be assigned to a variable)
Can contain multiple statements	Only one expression
Uses return	Returns the expression automatically
Better for complex logic	Best for short, simple functions

Syntax → `lambda arguments: expression`

Example 1 — def vs lambda (Add two numbers)

```
def add(x, y):
    return x + y
print(add(3, 5))           # using def

add = lambda x, y: x + y
print(add(3, 5))           # using lambda
```

Output: 8 | 8

Example 2 — Square & Double a number

```
square = lambda x: x ** 2
print(square(4))

double = lambda x: x * 2
print(double(10))
```

Output: 16 | 20

Example 3 — lambda combined with map(), filter(), reduce()

```
numbers = [1, 2, 3, 4, 5]

result = map(lambda x: x * 2, numbers)
print(list(result))           # map

result = filter(lambda x: x % 2 == 0, numbers)
print(list(result))           # filter

from functools import reduce
result = reduce(lambda x, y: x + y, numbers)
print(result)                 # reduce
```

Output: [2, 4, 6, 8, 10] | [2, 4] | 15

💡 **Tip:** Use lambda for quick, one-line logic. If your logic needs multiple steps or is reused often, define a normal function with `def` instead.

⇒ map() Function

`map()` applies a given function to **every item** in an iterable (list, tuple, set, etc.) and returns a **map object** (an iterator) — great for transforming data.

Function	Purpose
<code>map(function, iterable)</code>	Apply a function to every element of an iterable
Return value	A map object (iterator)
Common use	Transform data — multiply numbers, convert types, change text case

Example 1 — Multiply each element by 2

```
numbers = [1, 2, 3, 4, 5]
result = map(lambda x: x * 2, numbers)
print(list(result))
```

Output: [2, 4, 6, 8, 10]

Example 2 — Convert strings to integers & names to uppercase

```
numbers = ["1", "2", "3", "4"]
print(list(map(int, numbers)))

names = ["ali", "sara", "john"]
print(list(map(str.upper, names)))
```

Output: [1, 2, 3, 4] | ['ALI', 'SARA', 'JOHN']

Example 3 — Using a normal function

```
def square(x):
    return x ** 2

numbers = [1, 2, 3, 4, 5]
result = map(square, numbers)
print(list(result))
```

Output: [1, 4, 9, 16, 25]

Example 4 — map() returns an iterator, not a list

```
numbers = [1, 2, 3]
result = map(lambda x: x + 1, numbers)
print(result)           # <map object at 0x...>
print(list(result))     # [2, 3, 4]
```

Example 5 — Real-world: add 15% tax to prices

```
prices = [100, 200, 300, 400]
final_prices = list(map(lambda x: x * 1.15, prices))
print(final_prices)
```

Output: [115.0, 230.0, 345.0, 460.0]

💡 **Tip:** Always wrap `map()` in `list()` (or another container) to see or use the actual results — otherwise you just get an iterator object.

🔗 filter() Function

`filter()` selects only the elements of an iterable that satisfy a **condition** (a function that returns True or False) — useful for keeping only the data you need.

Syntax → `filter(function, iterable)`

Example 1 — Keep only even numbers

```
numbers = [1, 2, 3, 4, 5, 6]
result = filter(lambda x: x % 2 == 0, numbers)
print(list(result))
```

Output: [2, 4, 6]

Example 2 — Keep numbers greater than 10

```
numbers = [5, 12, 8, 20, 15]
result = filter(lambda x: x > 10, numbers)
print(list(result))
```

Output: [12, 20, 15]

Example 3 — Keep names that start with "A"

```
names = ["Ali", "Sara", "Ahmed", "John", "Adam"]
result = filter(lambda name: name.startswith("A"), names)
print(list(result))
```

Output: ['Ali', 'Ahmed', 'Adam']

Example 4 — Using a normal function

```
def is_even(x):
    return x % 2 == 0

numbers = [1, 2, 3, 4, 5, 6]
result = filter(is_even, numbers)
print(list(result))
```

Output: [2, 4, 6]

Example 5 — Real-world: students who passed (score ≥ 50)

```
scores = [45, 70, 30, 90, 60, 40]
passed = list(filter(lambda score: score >= 50, scores))
print(passed)
```

Output: [70, 90, 60]

💡 **Tip:** Unlike `map()`, the function passed to `filter()` must always return a boolean — elements where it's True are kept.

Σ reduce() Function

`reduce()` combines all elements of an iterable into a **single value** by repeatedly applying a function to pairs of elements, until only one result remains. It lives in the `functools` module.

Syntax →

```
from functools import reduce
reduce(function, iterable)
```

Example 1 — Sum all numbers

```
from functools import reduce
numbers = [1, 2, 3, 4, 5]
result = reduce(lambda x, y: x + y, numbers)
print(result)
```

Output: 15

Example 2 — Multiply all numbers

```
numbers = [1, 2, 3, 4]
result = reduce(lambda x, y: x * y, numbers)
print(result)
```

Output: 24

Example 3 — Find the largest number

```
numbers = [8, 3, 12, 5, 9]
largest = reduce(lambda x, y: x if x > y else y, numbers)
print(largest)
```

Output: 12

Example 4 — Real-world: total shopping bill

```
prices = [120, 250, 80, 150]
total = reduce(lambda x, y: x + y, prices)
print(total)
```

Output: 600

💡 **Tip:** `reduce()` always needs an import from `functools` — unlike `map()` and `filter()`, which are built-in.

Quick Recap

- **lambda** — write small anonymous functions inline
- **map()** — transform every element of an iterable
- **filter()** — keep only elements that satisfy a condition
- **reduce()** — combine all elements into one final value